Iterator

Season: 3 · Behavioral · Category: GoF · Level: Intermediate

Intent

- Access aggregates sequentially without exposing representation.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- TV channel remote next/prev.

When to Use

- See Intent.
- The design force named in Intent shows up more than once.

When Not to Use

- Overkill for one-off behavior.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator;

import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

/** Custom iterator that cycles through a non-empty list. */
public final class RoundRobin<T> implements Iterable<T> {
    private final List<T> items;

    public RoundRobin(List<T> items) {
        if (items.isEmpty()) throw new IllegalArgumentException("empty");
        this.items = List.copyOf(items);
    }

    @Override
    public Iterator<T> iterator() {
        return new Iterator<>() {
            int i = 0;
            public boolean hasNext() { return true; }
            public T next() {
                if (items.isEmpty()) throw new NoSuchElementException();
                var v = items.get(i % items.size());
                i++;
                return v;
            }
        };
    }
```

Classic Implementation (2)

```
}
```

Modern Java 25 / Functional

- Java 25 pattern matching often replaces Visitor/Interpreter boilerplate.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: gof/behavioral/iterator/
```

Pitfalls

- Prefer sealed + switch over classic Visitor in Java 25.
- Strategy as a function value.
- Observer: unsubscribe to avoid leaks.

Interview Q&A

- Q: Key interview angle for Iterator? A: Name the force (undo, open/closed, open for extension) and a modern alternative.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Access aggregates sequentially without exposing representation.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: TV channel remote next/prev. [0:40] Intent — State the design force in one sentence: Access aggregat...